Why do generative policies fail where ACT works? A diagnostic study of imitation learning on a low-cost bimanual robot

Louis Barthez

Research internship at AIST Tokyo Waterfront (April – July 2026), supervised by Shunki Itadera · Polytech Montpellier

Technical report · code: github.com/hcp-ai-hub/openarm-box-sorting · data & models: huggingface.co/Louislerikiki

Abstract. We bring up a bimanual 7-DoF OpenArm platform from an unwired kit to a complete teleoperation → recording → training → real-robot evaluation pipeline built on LeRobot, and use it to compare four imitation-learning policies on the same hardware and data: ACT, Diffusion Policy, VQ-BeT and the 4B-parameter vision-language-action model π0. The task is sorting boxes by weight, inferred from joint torque, with a hand-to-hand transfer when the target bin is on the opposite side. Trained on 100 teleoperated episodes (~249k frames, six RGB-D streams), ACT completes the full task end to end with imprecise grasps, while the three generative policies all fail with the same erratic, bursty motion. Rather than stopping at this observation, we test candidate causes one by one. Data volume, normalization mismatch, camera drift and hardware or control-loop timing are ruled out, and we identify a silent weight-loading bug in a generic model-construction path. An offline dry-run replay shows that the erratic behaviour originates inside the models: VQ-BeT produces large action jumps on 12% of frames of a smooth real episode, against 0% for ACT. We release the dataset, trained policies and diagnostic tooling.

1 Introduction

Imitation learning from teleoperated demonstrations is now a standard way to teach manipulation skills to robots, and low-cost bimanual hardware has made it accessible to small labs [1]. Moving from a published architecture to a policy that actually works on a new platform remains difficult: hardware integration, data collection and evaluation all interact, and when a policy fails on the real robot it is often unclear whether the model, the data or the hardware is responsible.

This report documents such a study on OpenArm, an open-source bimanual arm [6]. Its contributions are: (i) a complete, reusable record–train–evaluate pipeline on OpenArm with LeRobot [4]; (ii) a comparison of four policy families on identical hardware, task and data; (iii) a systematic diagnostic of a shared failure mode of the generative policies, including an offline replay test that separates model behaviour from hardware effects.

2 Platform and data

Hardware bring-up

The platform is two 7-DoF OpenArm follower arms with parallel grippers, driven by backdrivable quasi-direct-drive motors, plus two identical leader arms. Starting from an assembled but unwired kit, each arm was given its own CAN FD bus (1 Mbit/s nominal, 5 Mbit/s data phase), motor IDs were assigned one by one, and each joint was zeroed on a calibration jig.

Teleoperation

Leader–follower teleoperation was implemented first with low-level CAN scripts, in unilateral mode (joint positions streamed to the follower) and bilateral mode (follower torque fed back to the leader). Logging the torque on joint 4 during a repeated curl motion confirmed that the leader torque tracks the follower only in bilateral mode. Bilateral control also proved a safety feature during long recording sessions: the operator feels contacts that would otherwise silently damage the hardware. The same functionality was then reproduced with LeRobot's calibration and teleoperation tools.

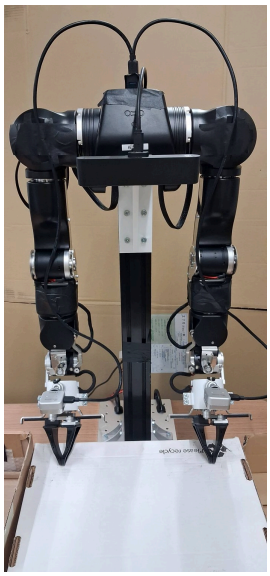


Figure 1. The OpenArm follower arms with the wrist and top cameras on custom 3D-printed mounts.

Task and dataset

The operator controls both leader arms to locate a box among several, grasp it, transfer it to the other arm when needed, and place it in the bin matching its weight. No weight sensor is used: weight is inferred from joint torque during the grasp. The final dataset contains 100

episodes (~249k frames at 30 fps) with box positions varied across the workspace. Each frame stores per-joint position, velocity and torque for both arms, the action, and six camera streams: an overhead ZED (RGB + depth) and two wrist RealSense cameras (RGB + depth). Off-the-shelf mounts either did not fit the arm geometry or blocked the gripper, so three camera mounts were designed and 3D-printed (Figure 1).

The pipeline was first validated with ACT on three simpler tasks using a single top camera: single-arm pick and drop, fixed-position hand-to-hand transfer, and single-arm sort by weight. All three worked reliably before the full task was attempted.

3 Policies and evaluation protocol

ACT [1] is a conditional VAE that predicts a chunk of future actions in one forward pass; it is the standard baseline for this class of hardware. **Diffusion Policy** [3] iteratively denoises an action sequence and is known for representing multimodal action distributions. **VQ-BeT** [2] predicts discrete action codes from a VQ-VAE codebook, chosen to test whether discretization avoids the averaging of several valid demonstrations. **π0** [5] is a ~4B-parameter vision-language-action model fine-tuned with flow matching; it had to be run in reduced precision to fit GPU memory, which exposed a chain of dtype mismatches in the forward pass that were patched one by one. Training ran on 4× RTX 6000 Ada GPUs.

Physical evaluation: each policy runs on the robot with the box at varied positions; we record grasp success, transfer success and correct bin, and characterize *how* a policy fails. **Dry-run evaluation**: a real recorded episode is replayed frame by frame into the policy without commanding the robot, and we measure the change of predicted action between consecutive frames. Since the input evolves smoothly, a well-behaved policy should output smoothly evolving actions; large jumps under these controlled conditions point to the model rather than the robot.

4 Results

Policy	Behaviour on the real robot
ACT	Completes the full task: finds the box, transfers it when needed, correct bin in most trials. Grasp alignment imprecise.
Diffusion Policy	Arm largely limp, sudden disconnected bursts, no directed approach.
VQ-BeT	Same pattern.
π0 (fine-tuned)	Same pattern once running.



Figure 2. ACT on the full bimanual task (video: `Louislerikiki/act_sort_by_weight_multicam_v1`).

To test whether ACT's imprecision came from too much variation between demonstrations, ACT was retrained on 25, 50 and 75 of the 100 episodes. Fewer episodes made the policy *more* erratic and the transfer less reliable, ruling out data volume as the main bottleneck. ACT variants (80% state dropout, four decoder layers, RGB only) changed grasp precision or attention focus at the margin but not the overall picture.

5 Diagnosing the shared failure

Three architectures built on very different mechanisms (denoising, discrete codes, pretrained flow matching) failing *in the same way* suggests a cause shared by the whole pipeline rather than by one model. We tested the candidates one at a time (Table 1).

Table 1. Hypotheses tested.

Hypothesis	Test	Outcome
Too much / too varied data	ACT on 25/50/75 episodes	Ruled out (fewer is worse)
Normalization mismatch	Policy stats vs dataset stats	Ruled out (exact match)
Camera drift between sessions	Pixel diff of first frames, early / mid / late episodes	Ruled out (negligible)
Weights not loaded	Generic builder vs policy-specific loader	**Real bug**: generic path silently skipped trained weights; all diagnostics re-run after the fix
Latency / control loop	Offline dry-run replay	Ruled out: jumps occur inside the model

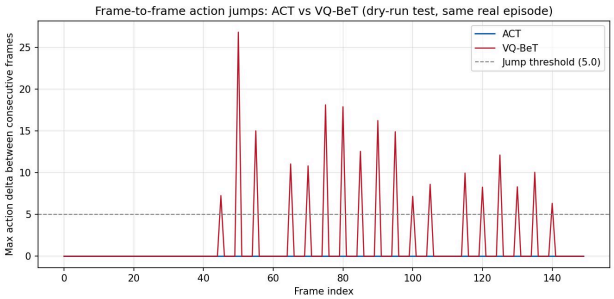


Figure 3. Dry-run test on the same 150-frame real episode: maximum action change between consecutive frames. ACT stays below the threshold throughout; VQ-BeT exceeds it on 18/150 frames (12%).

The dry-run test (Figure 3) is the key result: with no robot in the loop and a smoothly changing real input, VQ-BeT still produces large, repeated action jumps, while ACT produces none. The erratic behaviour is therefore internal to the model and not caused by latency, control frequency or hardware.

For ACT, a further test compared the predicted reach target with the true recorded target across episodes, at several points before the grasp. The correlation was weak and of inconsistent sign across joints: ACT's imprecision is not noise around a correct direction but a difficulty in mapping a specific visual input to a specific target.

6 Discussion and open leads

The most robust outcome is negative but informative: neither more nor less data, nor three very different generative architectures, resolved grasp precision on this platform. This rules out the most common explanations and points to what was held constant across all experiments: the camera setup, the action representation shared by all policies, and the demonstrations themselves. Two directions remain untested: (i) **camera auto-exposure**, never explicitly fixed, which may change lighting between sessions; (ii) **architectural changes** such as feeding force/torque to the policy or using a more constrained action representation to handle multimodal demonstrations.

Beyond the policies, the main deliverable is the pipeline and the dry-run method, which let future hypotheses be tested without physical access to the robot. Dataset, trained models and scripts are public.

Acknowledgements

I thank Shunki Itadera and the robotics team at AIST Tokyo Waterfront for their support throughout this internship.

References

[1] T. Z. Zhao, V. Kumar, S. Levine, C. Finn. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. arXiv:2304.13705, 2023.

[2] S. Lee, Y. Wang, H. Etukuru, H. J. Kim, N. M. M. Shafiullah, L. Pinto. Behavior Generation with Latent Actions (VQ-BeT). arXiv:2403.03181, 2024.

[3] C. Chi et al. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. RSS, 2023.

[4] Hugging Face. LeRobot: State-of-the-art Machine Learning for Real-World Robotics in PyTorch. github.com/huggingface/lerobot, 2024.

[5] K. Black et al. π0: A Vision-Language-Action Flow Model for General Robot Control. arXiv:2410.24164, 2024.

[6] Enactic. OpenArm documentation. docs.openarm.dev.